

```

void MmwDemo_transmitProcessedOutput(UART_Handle uartHandle,
                                     DPC_ObjectDetection_ExecuteResult
*result,
                                     MmwDemo_output_message_stats
*timingInfo)
{
    MmwDemo_output_message_header header;
    MmwDemo_GuiMonSel *pGuiMonSel;
    MmwDemo_SubFrameCfg *subFrameCfg;
    uint32_t tlvIdx = 0;
    uint32_t i;
    uint32_t numPaddingBytes;
    uint32_t packetLen;
    uint8_t padding[MMWDEMO_OUTPUT_MSG_SEGMENT_LEN];
    uint16_t *detMatrix = (uint16_t *)result->detMatrix.data;

    MmwDemo_output_message_t1 t1[MMWDEMO_OUTPUT_MSG_MAX];

    /* Get subframe configuration */
    subFrameCfg = &gMmwMCB.subFrameCfg[result->subFrameIdx];

    /* Get Gui Monitor configuration */
    pGuiMonSel = &subFrameCfg->guiMonSel;

    /* Clear message header */
    memset((void *)&header, 0, sizeof(MmwDemo_output_message_header));
    /* Header: */
    header.platform = 0xA6843;
    header.magicWord[0] = 0x0102;
    header.magicWord[1] = 0x0304;
    header.magicWord[2] = 0x0506;
    header.magicWord[3] = 0x0708;
    header.numDetectedObj = result->numObjOut;
    header.version = MMWAVE_SDK_VERSION_BUILD | //DEBUG_VERSION
                    (MMWAVE_SDK_VERSION_BUGFIX << 8) |
                    (MMWAVE_SDK_VERSION_MINOR << 16) |
                    (MMWAVE_SDK_VERSION_MAJOR << 24);

    packetLen = sizeof(MmwDemo_output_message_header);
    if ((pGuiMonSel->detectedObjects == 1) || (pGuiMonSel->detectedObjects
== 2) &&
        (result->numObjOut > 0))

```

```

    {
        tlv[tlvIdx].type = MMWDEMO_OUTPUT_MSG_DETECTED_POINTS;
        tlv[tlvIdx].length = sizeof(DPIF_PointCloudCartesian) *
result->numObjOut;
        packetLen += sizeof(MmwDemo_output_message_tlv) + tlv[tlvIdx].length;
        tlvIdx++;
    }
    /* Side info */
    if ((pGuiMonSel->detectedObjects == 1) && (result->numObjOut > 0))
    {
        tlv[tlvIdx].type = MMWDEMO_OUTPUT_MSG_DETECTED_POINTS_SIDE_INFO;
        tlv[tlvIdx].length = sizeof(DPIF_PointCloudSideInfo) *
result->numObjOut;
        packetLen += sizeof(MmwDemo_output_message_tlv) + tlv[tlvIdx].length;
        tlvIdx++;
    }
    if (pGuiMonSel->logMagRange)
    {
        tlv[tlvIdx].type = MMWDEMO_OUTPUT_MSG_RANGE_PROFILE;
        tlv[tlvIdx].length = sizeof(uint16_t) * subFrameCfg->numRangeBins;
        packetLen += sizeof(MmwDemo_output_message_tlv) + tlv[tlvIdx].length;
        tlvIdx++;
    }
    if (pGuiMonSel->noiseProfile)
    {
        tlv[tlvIdx].type = MMWDEMO_OUTPUT_MSG_NOISE_PROFILE;
        tlv[tlvIdx].length = sizeof(uint16_t) * subFrameCfg->numRangeBins;
        packetLen += sizeof(MmwDemo_output_message_tlv) + tlv[tlvIdx].length;
        tlvIdx++;
    }
    if (pGuiMonSel->rangeAzimuthHeatMap)
    {
#ifdef USE_2D_AOA_DPU
        tlv[tlvIdx].type =
MMWDEMO_OUTPUT_MSG_AZIMUT_ELEVATION_STATIC_HEAT_MAP;
#else
        tlv[tlvIdx].type = MMWDEMO_OUTPUT_MSG_AZIMUT_STATIC_HEAT_MAP;
#endif
        tlv[tlvIdx].length = result->azimuthStaticHeatMapSize *
sizeof(cmplx16ImRe_t);
        packetLen += sizeof(MmwDemo_output_message_tlv) +
tlv[tlvIdx].length;
        tlvIdx++;
    }

```

```

    }
    if (pGuiMonSel->rangeDopplerHeatMap)
    {
        tlv[tlvIdx].type = MMWDEMO_OUTPUT_MSG_RANGE_DOPPLER_HEAT_MAP;
        tlv[tlvIdx].length = subFrameCfg->numRangeBins *
subFrameCfg->numDopplerBins * sizeof(uint16_t);
        packetLen += sizeof(MmwDemo_output_message_tlv) + tlv[tlvIdx].length;
        tlvIdx++;
    }
    if (pGuiMonSel->statsInfo)
    {
        tlv[tlvIdx].type = MMWDEMO_OUTPUT_MSG_STATS;
        tlv[tlvIdx].length = sizeof(MmwDemo_output_message_stats);
        packetLen += sizeof(MmwDemo_output_message_tlv) + tlv[tlvIdx].length;
        tlvIdx++;

        MmwDemo_getTemperatureReport();
        tlv[tlvIdx].type = MMWDEMO_OUTPUT_MSG_TEMPERATURE_STATS;
        tlv[tlvIdx].length = sizeof(MmwDemo_temperatureStats);
        packetLen += sizeof(MmwDemo_output_message_tlv) + tlv[tlvIdx].length;
        tlvIdx++;
    }

    /* Fill header */
    header.numTLVs = tlvIdx;
    /* Round up packet length to multiple of MMWDEMO_OUTPUT_MSG_SEGMENT_LEN
*/
    header.totalPacketLen = MMWDEMO_OUTPUT_MSG_SEGMENT_LEN *
        ((packetLen +
(MMWDEMO_OUTPUT_MSG_SEGMENT_LEN-1))/MMWDEMO_OUTPUT_MSG_SEGMENT_LEN);
    header.timeCpuCycles = Pmu_getCount(0);
    header.frameNumber = result->stats->frameStartIntCounter;
    header.subFrameNumber = result->subFrameIdx;

    UART_writePolling (uartHandle,
        (uint8_t*)&header,
        sizeof(MmwDemo_output_message_header));

    tlvIdx = 0;

    for (i = 0; i < result->numObjOut; i++)
    {
        result->objOut[i].x = 1.0;
    }

```

```

        result->objOut[i].y = 2.0;
        result->objOut[i].z = 3.0;
    }
    /* Send detected Objects */
    if ((pGuiMonSel->detectedObjects == 1) || (pGuiMonSel->detectedObjects
== 2) &&
        (result->numObjOut > 0))
    {
        UART_writePolling (uartHandle,
                           (uint8_t*)&tl[tlvIdx],
                           sizeof(MmwDemo_output_message_tl));

        /*Send array of objects */
        UART_writePolling (uartHandle, (uint8_t*)result->objOut,
                           sizeof(DPIF_PointCloudCartesian) *
result->numObjOut);
        tlvIdx++;
    }

    /* Send detected Objects Side Info */
    if ((pGuiMonSel->detectedObjects == 1) && (result->numObjOut > 0))
    {
        UART_writePolling (uartHandle,
                           (uint8_t*)&tl[tlvIdx],
                           sizeof(MmwDemo_output_message_tl));

        /*Send array of objects */
        UART_writePolling (uartHandle, (uint8_t*)result->objOutSideInfo,
                           sizeof(DPIF_PointCloudSideInfo) *
result->numObjOut);
        tlvIdx++;
    }

    /* Send Range profile */
    if (pGuiMonSel->logMagRange)
    {
        UART_writePolling (uartHandle,
                           (uint8_t*)&tl[tlvIdx],
                           sizeof(MmwDemo_output_message_tl));

        for(i = 0; i < subFrameCfg->numRangeBins; i++)
        {
            UART_writePolling (uartHandle,

```

```

        (uint8_t*)&detMatrix[i * subFrameCfg->numDopplerBins],
        sizeof(uint16_t));
    }
    tlvIdx++;
}

/* Send noise profile */
if (pGuiMonSel->noiseProfile)
{
    uint32_t maxDopIdx = subFrameCfg->numDopplerBins/2 -1;
    UART_writePolling (uartHandle,
        (uint8_t*)&tl[tlvIdx],
        sizeof(MmwDemo_output_message_tl));

    for(i = 0; i < subFrameCfg->numRangeBins; i++)
    {
        UART_writePolling (uartHandle,
            (uint8_t*)&detMatrix[i * subFrameCfg->numDopplerBins +
maxDopIdx],
            sizeof(uint16_t));
    }
    tlvIdx++;
}

/* Send data for static azimuth heatmap */
if (pGuiMonSel->rangeAzimuthHeatMap)
{
    UART_writePolling (uartHandle,
        (uint8_t*)&tl[tlvIdx],
        sizeof(MmwDemo_output_message_tl));

    UART_writePolling (uartHandle,
        (uint8_t *) result->azimuthStaticHeatMap,
        result->azimuthStaticHeatMapSize * sizeof(cmplx16ImRe_t));

    tlvIdx++;
}

/* Send data for range/Doppler heatmap */
if (pGuiMonSel->rangeDopplerHeatMap == 1)
{
    UART_writePolling (uartHandle,
        (uint8_t*)&tl[tlvIdx],

```

```

        sizeof(MmwDemo_output_message_t1));
    UART_writePolling (uartHandle,
        (uint8_t*)detMatrix,
        t1[tlvIdx].length);
    tlvIdx++;
}

/* Send stats information */
if (pGuiMonSel->statsInfo == 1)
{
    UART_writePolling (uartHandle,
        (uint8_t*)&t1[tlvIdx],
        sizeof(MmwDemo_output_message_t1));
    UART_writePolling (uartHandle,
        (uint8_t*)timingInfo,
        t1[tlvIdx].length);

    tlvIdx++;
    UART_writePolling (uartHandle,
        (uint8_t*)&t1[tlvIdx],
        sizeof(MmwDemo_output_message_t1));
    UART_writePolling (uartHandle,
        (uint8_t*)&gMmwMCB.temperatureStats,
        t1[tlvIdx].length);

    tlvIdx++;
}

/* Send padding bytes */
numPaddingBytes = MMWDEMO_OUTPUT_MSG_SEGMENT_LEN - (packetLen &
(MMWDEMO_OUTPUT_MSG_SEGMENT_LEN-1));
if (numPaddingBytes<MMWDEMO_OUTPUT_MSG_SEGMENT_LEN)
{
    UART_writePolling (uartHandle,
        (uint8_t*)padding,
        numPaddingBytes);
}
}

```